Beginner's Guideline: Autonomous Navigation & Vision (TurtleBot4) Task

This is a learning path for the recruitment task: connect to a WebSocket telemetry server, drive a TurtleBot4 through Gazebo to a set of waypoints using direct `/cmd_vel` commands, then use OpenCV to identify the color of the largest of three spheres. NAV2 and SLAM are optional bonus to attempt only after the core cmd_vel navigation and vision pipeline both work.

Most of the ROS 2/Gazebo basics are well covered by standard tutorials, so beginners can move quickly through those sections and spend extra time on WebSocket integration and sphere color-segmentation — these two are the least-covered-by-standard-tutorials parts of the task.

---

1. ROS 2 Fundamentals (skim if already comfortable)

- **Official docs:** docs.ros.org — ROS 2 Humble/Jazzy Tutorials — nodes, topics, publishers/subscribers, parameters, launch files.
- **YouTube:** *"ROS 2 Tutorial for Beginners"* — The Construct's ROS 2 basics playlist, and Articulated Robotics' *"Building a mobile robot"* series (excellent for tying URDF + Gazebo + control together).
- The part most beginners haven't seen yet is `rclpy` node structure with **timers + multiple subscriptions running concurrently** (odom + scan + your own control loop), and writing a simple go-to-goal controller by hand (distance/heading error → `/cmd_vel` twist) instead of relying on a planner. The provided `waypoint_follower.py` demonstrates exactly this pattern — read through it once before writing your own version.

2. Gazebo + TurtleBot4 Simulation Setup

- **Official TurtleBot4 User Manual — Simulation:** https://turtlebot.github.io/turtlebot4-user-manual/software/simulation.html
- **Gazebo (Ignition/Harmonic) official tutorials:** https://gazebosim.org/docs/harmonic/tutorials/
- **YouTube:** Clearpath Robotics' official TurtleBot4 channel has setup and simulation walkthroughs; search *"TurtleBot4 Ignition Gazebo simulation tutorial"* for current Humble/Jazzy walkthroughs.
- Note: this task does **not** use Nav2 or a planner — you drive the robot yourself by publishing directly to `/cmd_vel` based on odometry feedback, so you can skip Nav2-specific tutorials entirely for the core task.
- Practical step: clone `SadmanSyfe/Recruitment-task`, read the `main_assignment.launch.py` file to see exactly which nodes/topics/services it brings up

3. WebSocket Telemetry Client (usually the least familiar part)

The task wants a ROS 2 node that also runs a WebSocket client to `ws://localhost:8765` and parses JSON waypoints — this is not a standard ROS 2 tutorial topic, so here's the direct path:

- **Library:** Python's `websockets` library (async) is the standard choice: https://websockets.readthedocs.io/en/stable/
- **Docs — quickstart:** https://websockets.readthedocs.io/en/stable/reference/client.html
- **YouTube:** Search *"Python websockets client tutorial asyncio"* — any recent (2023+) video covering the `websockets` package's async client API works; the API is stable.
- **Key integration challenge:** `rclpy` normally spins on its own executor, while `websockets` needs an `asyncio` event loop. Two common patterns:
  1. Run the WebSocket client in a separate thread using `asyncio.run()`, and push received waypoints into a thread-safe queue that your ROS timer callback drains.
  2. Use `rclpy`'s executor alongside `asyncio` via a small bridging loop (more advanced — thread approach is simpler and fine for this task).
- Search *"rclpy asyncio integration"* if you want to see how others have bridged the two; there's no single canonical doc for this, so expect to piece it together from a couple of blog posts/GitHub issues.

4. Computer Vision — Sphere Segmentation & Color ID

- **OpenCV official docs — color spaces & thresholding:** https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
- **HSV-based color detection tutorial:** https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html
- **Contour detection & finding the largest contour:** https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html
- **YouTube:** *"OpenCV color detection HSV Python"* (Murtaza's Workshop / freeCodeCamp OpenCV course both cover this clearly) and *"OpenCV Hough Circle detection"* if you want to detect the spheres by shape rather than just color blobs.
- **Suggested approach for this task:**
  1. Subscribe to the camera topic (check `main_assignment.launch.py`/`ros2 topic list` for the exact name) via `cv_bridge` to convert `sensor_msgs/Image` → OpenCV `numpy` array.
  2. Convert BGR → HSV.
  3. Threshold for each candidate sphere color (red/green/blue, or whatever the environment defines) with `cv2.inRange`.
  4. `cv2.findContours` on each mask, take the largest contour per color, compare areas across colors — the largest-area contour overall tells you which color the biggest sphere is.

5. Alternatively, `cv2.HoughCircles` to detect circular blobs first, then sample the HSV value at each detected circle's centroid — more robust if spheres overlap visually.

- **`cv_bridge` reference:** https://docs.ros.org/en/humble/p/cv_bridge/ — converts ROS `Image` messages to OpenCV Mats and back.

5. Optional Bonus — SLAM Integration (attempt only after the core task works)

This is a bonus, not a requirement — only look at this once waypoint navigation (via plain `/cmd_vel`) and the sphere color detection are both working reliably.

- **slam_toolbox official docs:** https://github.com/SteveMacenski/slam_toolbox (README covers async vs. sync mapping modes and the launch parameters).
- **YouTube:** Articulated Robotics' *"SLAM for a mobile robot (ROS2)"* video walks through `slam_toolbox` setup end-to-end and is one of the clearest independent tutorials available.
- In practice this is mostly a matter of launching `slam_toolbox` pointed at the TurtleBot4's `/scan` and `/odom` topics alongside your existing `waypoint_follower.py`, then confirming a map builds correctly in RViz while the robot drives to its waypoints. You don't need Nav2 for this — `slam_toolbox` builds the map independently of how the robot is being driven.

6. Further Optional — Nav2 (for anyone who wants to go beyond the task)

Not required at all for this task, but if you're curious how a "real" planner-based approach would work instead of hand-written go-to-goal control, Nav2 is the standard ROS 2 navigation stack and a natural next step once SLAM is working:

- **Official Nav2 docs:** https://docs.nav2.org/
- **Nav2 + SLAM tutorial:** https://docs.nav2.org/tutorials/docs/navigation2_with_slam.html
- **TurtleBot4 Navigator examples (waypoint following with Nav2):** https://turtlebot.github.io/turtlebot4-user-manual/tutorials/turtlebot4_navigator.html
- **TurtleBot4 Navigation tutorial:** https://turtlebot.github.io/turtlebot4-user-manual/tutorials/navigation.html
- **YouTube:** Articulated Robotics' *"Setting up Nav2 in ROS2"* series is a clear, current walkthrough.
- The idea: once you have a map from `slam_toolbox`, Nav2 takes over global/local path planning, costmap-based obstacle avoidance, and waypoint following via action clients — instead of you writing the go-to-goal math by hand. Good to know for future robotics work, but purely exploratory for this task.